

- [Step 1: Initialize Project & Install Dependencies](#)
- [Step 2: Configuration](#)
- [Step 3: Helper Utilities](#)
- [Step 4: Middleware](#)
- [Step 5: Models](#)
- [Step 6: Account Service Implementation](#)
- [Step 7: Accounts Controller](#)
- [Step 8: Final Server Wiring](#)
- [Step 9: Running and Verification](#)

To ensure you have the full code as detailed in the sources, follow this comprehensive guide. This version includes every specific function and configuration provided in the manual.

Step 1: Initialize Project & Install Dependencies

Create your project folder and initialize Node.js.

```
mkdir node-mysql-api
cd node-mysql-api
npm init -y
```

Install Production Dependencies:

```
npm install express cors body-parser cookie-parser bcryptjs jsonwebtoken mysql2
sequelize nodemailer joi swagger-ui-express yamls express-jwt
```

Install Development Dependencies:

```
npm install -D typescript ts-node nodemon @types/node @types/express @types/cors
@types/cookie-parser @types/bcryptjs @types/jsonwebtoken @types/nodemailer
@types/swagger-ui-express @types/yamls
```

Initialize TypeScript: Run `npx tsc --init` and update your `tsconfig.json` to match the core settings:

```
{
  "compilerOptions": {
    "target": "es2018",
    "module": "commonjs",
    "lib": ["es2018"],
    "outDir": "./dist",
    "rootDir": "./",
    "strict": false,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["**/*.ts"],
  "exclude": ["node_modules"]
}
```

Update `package.json` Scripts:

```
"scripts": {
  "start": "ts-node ./server.ts",
  "start:dev": "nodemon --exec ts-node ./server.ts"
}
```

Step 2: Configuration

Create `config.json` in your root directory. Use **Ethereal Email** for testing SMTP.

```
{
  "database": {
    "host": "localhost",
    "port": 3306,
    "user": "root",
    "password": "your_mysql_password",
    "database": "node_mysql_api"
  },
  "secret": "SUPER_SECRET_KEY_REPLACE_ME_IN_PRODUCTION",
  "emailFrom": "info@my-node-api.com",
  "smtpOptions": {
    "host": "smtp.ethereal.email",
    "port": 587,
    "auth": {
      "user": "your_ethereal_user",
      "pass": "your_ethereal_pass"
    }
  }
}
```

```
}  
}
```

Step 3: Helper Utilities

Create a folder named `_helpers`.

1. Database Connection (`_helpers/db.ts`):

```
import config from '../config.json';  
import mysql from 'mysql2/promise';  
import { Sequelize } from 'sequelize';  
import accountModel from '../accounts/account.model';  
import refreshTokenModel from '../accounts/refresh-token.model';  
  
const db: any = {};  
export default db;  
  
initialize();  
  
async function initialize() {  
  const { host, port, user, password, database } = config.database;  
  const connection = await mysql.createConnection({ host, port, user, password  
});  
  await connection.query(`CREATE DATABASE IF NOT EXISTS \`${database}\``);  
  
  const sequelize = new Sequelize(database, user, password, { dialect: 'mysql'  
});  
  
  db.Account = accountModel(sequelize);  
  db.RefreshToken = refreshTokenModel(sequelize);  
  
  db.Account.hasMany(db.RefreshToken, { onDelete: 'CASCADE' });  
  db.RefreshToken.belongsTo(db.Account);  
  
  await sequelize.sync();  
}
```

2. Email Sender (`_helpers/send-email.ts`):

```
import nodemailer from 'nodemailer';  
import config from '../config.json';  
  
export default async function sendEmail({ to, subject, html, from =  
config.emailFrom }: any) {  
  const transporter = nodemailer.createTransport(config.smtpOptions);
```

```
    await transporter.sendMail({ from, to, subject, html });
  }
```

3. Roles Enum (`_helpers/role.ts`):

```
export default {
  Admin: 'Admin',
  User: 'User'
}
```

4. Swagger Docs Route (`_helpers/swagger.ts`):

```
import express from 'express';
const router = express.Router();
import swaggerUi from 'swagger-ui-express';
import YAML from 'yamljs';

const swaggerDocument = YAML.load('./swagger.yaml');

router.use('/', swaggerUi.serve, swaggerUi.setup(swaggerDocument));

export default router;
```

Step 4: Middleware

Create a folder named `_middleware`.

1. Global Error Handler (`_middleware/error-handler.ts`):

```
import { Request, Response, NextFunction } from 'express';

export default function errorHandler(err: any, req: Request, res: Response, next: NextFunction) {
  switch (true) {
    case typeof err === 'string':
      const is404 = err.toLowerCase().endsWith('not found');
      const statusCode = is404 ? 404 : 400;
      return res.status(statusCode).json({ message: err });
    case err.name === 'UnauthorizedError':
      return res.status(401).json({ message: 'Unauthorized' });
    default:
      return res.status(500).json({ message: err.message });
  }
}
```

2. Request Validation Helper (`_middleware/validate-request.ts`):

```
export default validateRequest;

function validateRequest(req: any, next: any, schema: any) {
  const options = {
    abortEarly: false,
    allowUnknown: true,
    stripUnknown: true
  };
  const { error, value } = schema.validate(req.body, options);
  if (error) {
    next(`Validation error: ${error.details.map((x: any) => x.message).join(',')}
  `});
  } else {
    req.body = value;
    next();
  }
}
```

3. Authorization Middleware (`_middleware/authorize.ts`):

```
import jwt from 'express-jwt';
import config from '../config.json';
import db from '../_helpers/db';

const { secret } = config;

export default function authorize(roles: any = []) {
  if (typeof roles === 'string') {
    roles = [roles];
  }

  return [
    jwt({ secret, algorithms: ['HS256'] }),
    async (req: any, res: any, next: any) => {
      const account = await db.Account.findByPk(req.user.id);

      if (!account || (roles.length && !roles.includes(account.role))) {
        return res.status(401).json({ message: 'Unauthorized' });
      }

      req.user.role = account.role;
      const refreshTokens = await account.getRefreshTokens();
      req.user.ownsToken = (token: any) => !!refreshTokens.find((x: any) =>
x.token === token);
      next();
    }
  ];
}
```

Step 5: Models

Create an `accounts` folder.

1. Account Model (`accounts/account.model.ts`):

```
import { DataTypes } from 'sequelize';

export default function model(sequelize: any) {
  const attributes = {
    email: { type: DataTypes.STRING, allowNull: false },
    passwordHash: { type: DataTypes.STRING, allowNull: false },
    title: { type: DataTypes.STRING, allowNull: false },
    firstName: { type: DataTypes.STRING, allowNull: false },
    lastName: { type: DataTypes.STRING, allowNull: false },
    acceptTerms: { type: DataTypes.BOOLEAN },
    role: { type: DataTypes.STRING, allowNull: false },
    verificationToken: { type: DataTypes.STRING },
    verified: { type: DataTypes.DATE },
    resetToken: { type: DataTypes.STRING },
    resetTokenExpires: { type: DataTypes.DATE },
    passwordReset: { type: DataTypes.DATE },
    created: { type: DataTypes.DATE, allowNull: false, defaultValue:
DataTypes.NOW },
    updated: { type: DataTypes.DATE },
    isVerified: {
      type: DataTypes.VIRTUAL,
      get() { return !(this.verified || this.passwordReset); }
    }
  };

  const options = {
    timestamps: false,
    defaultScope: { attributes: { exclude: ['passwordHash'] } },
    scopes: { withHash: { attributes: {}, } }
  };

  return sequelize.define('account', attributes, options);
}
```

2. Refresh Token Model (`accounts/refresh-token.model.ts`):

```
import { DataTypes } from 'sequelize';

export default function model(sequelize: any) {
  const attributes = {
    token: { type: DataTypes.STRING },
    expires: { type: DataTypes.DATE },
    created: { type: DataTypes.DATE, allowNull: false, defaultValue:
DataTypes.NOW },
```

```

    createdByIp: { type: DataTypes.STRING },
    revoked: { type: DataTypes.DATE },
    revokedByIp: { type: DataTypes.STRING },
    replacedByToken: { type: DataTypes.STRING },
    isExpired: {
      type: DataTypes.VIRTUAL,
      get() { return Date.now() >= this.expires; }
    },
    isActive: {
      type: DataTypes.VIRTUAL,
      get() { return !this.revoked && !this.isExpired; }
    }
  };

  const options = { timestamps: false };
  return sequelize.define('refreshToken', attributes, options);
}

```

Step 6: Account Service Implementation

Create **accounts/account.service.ts**. This file handles core logic like **token rotation**.

```

import config from '../config.json';
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
import crypto from 'crypto';
import { Op } from 'sequelize';
import sendEmail from '../_helpers/send-email';
import db from '../_helpers/db';
import Role from '../_helpers/role';

export default {
  authenticate,
  refreshToken,
  revokeToken,
  register,
  verifyEmail,
  forgotPassword,
  validateResetToken,
  resetPassword,
  getAll,
  getById,
  create,
  update,
  delete: _delete
};

// --- IMPLEMENTATION FUNCTIONS ---

async function authenticate({ email, password, ipAddress }: any) {

```

```

    const account = await db.Account.scope('withHash').findOne({ where: { email }
});
    if (!account || !account.isVerified || !(await bcrypt.compare(password,
account.passwordHash))) {
        throw 'Email or password is incorrect';
    }

    const jwtToken = generateJwtToken(account);
    const refreshToken = generateRefreshToken(account, ipAddress);
    await refreshToken.save();

    return { ...basicDetails(account), jwtToken, refreshToken: refreshToken.token
};
}

async function refreshToken({ token, ipAddress }: any) {
    const refreshToken = await getRefreshToken(token);
    const account = await refreshToken.getAccount();

    const newRefreshToken = generateRefreshToken(account, ipAddress);
    refreshToken.revoked = Date.now();
    refreshToken.revokedByIp = ipAddress;
    refreshToken.replacedByToken = newRefreshToken.token;
    await refreshToken.save();
    await newRefreshToken.save();

    const jwtToken = generateJwtToken(account);
    return { ...basicDetails(account), jwtToken, refreshToken:
newRefreshToken.token };
}

async function revokeToken({ token, ipAddress }: any) {
    const refreshToken = await getRefreshToken(token);
    refreshToken.revoked = Date.now();
    refreshToken.revokedByIp = ipAddress;
    await refreshToken.save();
}

async function register(params: any, origin: any) {
    if (await db.Account.findOne({ where: { email: params.email } })) {
        return await sendAlreadyRegisteredEmail(params.email, origin);
    }

    const account = new db.Account(params);
    const isFirstAccount = (await db.Account.count()) === 0;
    account.role = isFirstAccount ? Role.Admin : Role.User;
    account.verificationToken = randomTokenString();
    account.passwordHash = await hash(params.password);
    await account.save();

    await sendVerificationEmail(account, origin);
}

async function verifyEmail({ token }: any) {
    const account = await db.Account.findOne({ where: { verificationToken: token }
});
    if (!account) throw 'Verification failed';
}

```

```

    account.verified = Date.now();
    account.verificationToken = null;
    await account.save();
}

async function forgotPassword({ email }: any, origin: any) {
    const account = await db.Account.findOne({ where: { email } });
    if (!account) return;
    account.resetToken = randomTokenString();
    account.resetTokenExpires = new Date(Date.now() + 24*60*60*1000);
    await account.save();
    await sendPasswordResetEmail(account, origin);
}

async function validateResetToken({ token }: any) {
    const account = await db.Account.findOne(
        where: { resetToken: token, resetTokenExpires: { [Op.gt]: Date.now() } }
    );
    if (!account) throw 'Invalid token';
    return account;
}

async function resetPassword({ token, password }: any) {
    const account = await validateResetToken({ token });
    account.passwordHash = await hash(password);
    account.passwordReset = Date.now();
    account.resetToken = null;
    await account.save();
}

async function getAll() {
    const accounts = await db.Account.findAll();
    return accounts.map((x: any) => basicDetails(x));
}

async function getById(id: any) {
    const account = await getAccount(id);
    return basicDetails(account);
}

async function create(params: any) {
    if (await db.Account.findOne({ where: { email: params.email } })) {
        throw 'Email "' + params.email + '" is already registered';
    }
    const account = new db.Account(params);
    account.verified = Date.now();
    account.passwordHash = await hash(params.password);
    await account.save();
    return basicDetails(account);
}

async function update(id: any, params: any) {
    const account = await getAccount(id);
    if (params.email && account.email !== params.email && await
    db.Account.findOne({ where: { email: params.email } })) {
        throw 'Email "' + params.email + '" is already taken';
    }
}

```

```

    if (params.password) {
        params.passwordHash = await hash(params.password);
    }
    Object.assign(account, params);
    account.updated = Date.now();
    await account.save();
    return basicDetails(account);
}

async function _delete(id: any) {
    const account = await getAccount(id);
    await account.destroy();
}

// --- HELPER FUNCTIONS ---

async function getAccount(id: any) {
    const account = await db.Account.findByPk(id);
    if (!account) throw 'Account not found';
    return account;
}

async function getRefreshToken(token: any) {
    const refreshToken = await db.RefreshToken.findOne({ where: { token } });
    if (!refreshToken || !refreshToken.isActive) throw 'Invalid token';
    return refreshToken;
}

async function hash(password: any) {
    return await bcrypt.hash(password, 10);
}

function generateJwtToken(account: any) {
    return jwt.sign({ sub: account.id, id: account.id }, config.secret, {
        expiresIn: '15m' });
}

function generateRefreshToken(account: any, ipAddress: any) {
    return new db.RefreshToken({
        accountId: account.id,
        token: randomTokenString(),
        expires: new Date(Date.now() + 7*24*60*60*1000),
        createdByIp: ipAddress
    });
}

function randomTokenString() {
    return crypto.randomBytes(40).toString('hex');
}

function basicDetails(account: any) {
    const { id, title, firstName, lastName, email, role, created, updated,
        isVerified } = account;
    return { id, title, firstName, lastName, email, role, created, updated,
        isVerified };
}

```

```

async function sendVerificationEmail(account: any, origin: any) {
  let message;
  if (origin) {
    const verifyUrl = `${origin}/account/verify-email?
token=${account.verificationToken}`;
    message = `

Please click the below link to verify your email address:</p>
<p><a href="${verifyUrl}">${verifyUrl}</a></p>`;
  } else {
    message = `

Please use the below token to verify your email address with
the <code>/account/verify-email</code> api route:</p>
<p><code>${account.verificationToken}</code></p>`;
  }

  await sendEmail({
    to: account.email,
    subject: 'Sign-up Verification API - Verify Email',
    html: `

#### Verify Email</h4>${message}` }); } async function sendAlreadyRegisteredEmail(email: any, origin: any) { let message; if (origin) { message = ` If you don't know your password please visit the <a href="${origin}/account/forgot-password">forgot password</a> page.</p>`; } else { message = ` If you don't know your password you can reset it via the <code>/account/forgot-password</code> api route.</p>`; } await sendEmail({ to: email, subject: 'Sign-up Verification API - Email Already Registered', html: `Email Already Registered</h4><p>Your email <strong>${email} </strong> is already registered.</p>${message}` }); } async function sendPasswordResetEmail(account: any, origin: any) { let message; if (origin) { const resetUrl = `${origin}/account/reset-password? token=${account.resetToken}`; message = ` Please click the below link to reset your password:</p> <p><a href="${resetUrl}">${resetUrl}</a></p>`; } else { message = ` Please use the below token to reset your password with the <code>/account/reset-password</code> api route:</p> <p><code>${account.resetToken}</code></p>`; } await sendEmail({ to: account.email, subject: 'Sign-up Verification API - Reset Password', html: `Reset Password Email</h4>${message}` }); }


```

Step 7: Accounts Controller

Create `accounts/accounts.controller.ts`. This defines your routes and request validation.

```
import express from 'express';
const router = express.Router();
import Joi from 'joi';
import validateRequest from '../_middleware/validate-request';
import authorize from '../_middleware/authorize';
import Role from '../_helpers/role';
import accountService from './account.service';

// --- ROUTES ---
router.post('/authenticate', authenticateSchema, authenticate);
router.post('/refresh-token', refreshToken);
router.post('/revoke-token', authorize(), revokeTokenSchema, revokeToken);
router.post('/register', registerSchema, register);
router.post('/verify-email', verifyEmailSchema, verifyEmail);
router.post('/forgot-password', forgotPasswordSchema, forgotPassword);
router.post('/validate-reset-token', validateResetTokenSchema, validateResetToken);
router.post('/reset-password', resetPasswordSchema, resetPassword);
router.get('/', authorize(Role.Admin), getAll);
router.get('/:id', authorize(), getById);
router.post('/', authorize(Role.Admin), createSchema, create);
router.put('/:id', authorize(), updateSchema, update);
router.delete('/:id', authorize(), _delete);

export default router;

// --- ROUTE FUNCTIONS ---

function authenticateSchema(req: any, res: any, next: any) {
  const schema = Joi.object({
    email: Joi.string().required(),
    password: Joi.string().required()
  });
  validateRequest(req, next, schema);
}

function authenticate(req: any, res: any, next: any) {
  const { email, password } = req.body;
  const ipAddress = req.ip;
  accountService.authenticate({ email, password, ipAddress })
    .then(({ refreshToken, ...account }: any) => {
      setTokenCookie(res, refreshToken);
      res.json(account);
    })
    .catch(next);
}
```

```

function refreshToken(req: any, res: any, next: any) {
  const token = req.cookies.refreshToken;
  const ipAddress = req.ip;
  accountService.refreshToken({ token, ipAddress })
    .then(({ refreshToken, ...account }: any) => {
      setTokenCookie(res, refreshToken);
      res.json(account);
    })
    .catch(next);
}

function revokeTokenSchema(req: any, res: any, next: any) {
  const schema = Joi.object({ token: Joi.string().empty('') });
  validateRequest(req, next, schema);
}

function revokeToken(req: any, res: any, next: any) {
  const token = req.body.token || req.cookies.refreshToken;
  const ipAddress = req.ip;
  if (!token) return res.status(400).json({ message: 'Token is required' });
  if (!req.user.ownsToken(token) && req.user.role !== Role.Admin) {
    return res.status(401).json({ message: 'Unauthorized' });
  }
  accountService.revokeToken({ token, ipAddress })
    .then(() => res.json({ message: 'Token revoked' }))
    .catch(next);
}

function registerSchema(req: any, res: any, next: any) {
  const schema = Joi.object({
    title: Joi.string().required(),
    firstName: Joi.string().required(),
    lastName: Joi.string().required(),
    email: Joi.string().email().required(),
    password: Joi.string().min(6).required(),
    confirmPassword: Joi.string().valid(Joi.ref('password')).required(),
    acceptTerms: Joi.boolean().valid(true).required()
  });
  validateRequest(req, next, schema);
}

function register(req: any, res: any, next: any) {
  accountService.register(req.body, req.get('origin'))
    .then(() => res.json({ message: 'Registration successful, please check your email' }))
    .catch(next);
}

function verifyEmailSchema(req: any, res: any, next: any) {
  const schema = Joi.object({ token: Joi.string().required() });
  validateRequest(req, next, schema);
}

function verifyEmail(req: any, res: any, next: any) {
  accountService.verifyEmail(req.body)
    .then(() => res.json({ message: 'Verification successful' }))
    .catch(next);
}

```

```

}

function forgotPasswordSchema(req: any, res: any, next: any) {
  const schema = Joi.object({ email: Joi.string().email().required() });
  validateRequest(req, next, schema);
}

function forgotPassword(req: any, res: any, next: any) {
  accountService.forgotPassword(req.body, req.get('origin'))
    .then(() => res.json({ message: 'Please check your email for password reset instructions' }))
    .catch(next);
}

function validateResetTokenSchema(req: any, res: any, next: any) {
  const schema = Joi.object({ token: Joi.string().required() });
  validateRequest(req, next, schema);
}

function validateResetToken(req: any, res: any, next: any) {
  accountService.validateResetToken(req.body)
    .then(() => res.json({ message: 'Token is valid' }))
    .catch(next);
}

function resetPasswordSchema(req: any, res: any, next: any) {
  const schema = Joi.object({
    token: Joi.string().required(),
    password: Joi.string().min(6).required(),
    confirmPassword: Joi.string().valid(Joi.ref('password')).required()
  });
  validateRequest(req, next, schema);
}

function resetPassword(req: any, res: any, next: any) {
  accountService.resetPassword(req.body)
    .then(() => res.json({ message: 'Password reset successful' }))
    .catch(next);
}

function getAll(req: any, res: any, next: any) {
  accountService.getAll()
    .then(accounts => res.json(accounts))
    .catch(next);
}

function getById(req: any, res: any, next: any) {
  if (Number(req.params.id) !== req.user.id && req.user.role !== Role.Admin) {
    return res.status(401).json({ message: 'Unauthorized' });
  }
  accountService.getById(req.params.id)
    .then(account => account ? res.json(account) : res.sendStatus(404))
    .catch(next);
}

function createSchema(req: any, res: any, next: any) {
  const schema = Joi.object({

```

```

        title: Joi.string().required(),
        firstName: Joi.string().required(),
        lastName: Joi.string().required(),
        email: Joi.string().email().required(),
        password: Joi.string().min(6).required(),
        confirmPassword: Joi.string().valid(Joi.ref('password')).required(),
        role: Joi.string().valid(Role.Admin, Role.User).required()
    });
    validateRequest(req, next, schema);
}

function create(req: any, res: any, next: any) {
    accountService.create(req.body)
        .then(account => res.json(account))
        .catch(next);
}

function updateSchema(req: any, res: any, next: any) {
    const schemaRules: any = {
        title: Joi.string().empty(''),
        firstName: Joi.string().empty(''),
        lastName: Joi.string().empty(''),
        email: Joi.string().email().empty(''),
        password: Joi.string().min(6).empty(''),
        confirmPassword: Joi.string().valid(Joi.ref('password')).empty('')
    };

    if (req.user.role === Role.Admin) {
        schemaRules.role = Joi.string().valid(Role.Admin, Role.User).empty('');
    }

    const schema = Joi.object(schemaRules).with('password', 'confirmPassword');
    validateRequest(req, next, schema);
}

function update(req: any, res: any, next: any) {
    if (Number(req.params.id) !== req.user.id && req.user.role !== Role.Admin) {
        return res.status(401).json({ message: 'Unauthorized' });
    }
    accountService.update(req.params.id, req.body)
        .then(account => res.json(account))
        .catch(next);
}

function _delete(req: any, res: any, next: any) {
    if (Number(req.params.id) !== req.user.id && req.user.role !== Role.Admin) {
        return res.status(401).json({ message: 'Unauthorized' });
    }
    accountService.delete(req.params.id)
        .then(() => res.json({ message: 'Account deleted successfully' }))
        .catch(next);
}

function setTokenCookie(res: any, token: any) {
    const cookieOptions = {
        httpOnly: true,
        expires: new Date(Date.now() + 7*24*60*60*1000)
    };

```

```
};  
res.cookie('refreshToken', token, cookieOptions);  
}
```

Step 8: Final Server Wiring

Create **server.ts** in the root folder.

```
import express from 'express';  
import bodyParser from 'body-parser';  
import cookieParser from 'cookie-parser';  
import cors from 'cors';  
import errorHandler from './_middleware/error-handler';  
import accountsController from './accounts/accounts.controller';  
import swaggerDocs from './_helpers/swagger';  
  
const app = express();  
  
app.use(bodyParser.urlencoded({ extended: false }));  
app.use(bodyParser.json());  
app.use(cookieParser());  
  
app.use(cors({ origin: (origin, callback) => callback(null, true), credentials:  
true }));  
  
app.use('/accounts', accountsController);  
app.use('/api-docs', swaggerDocs);  
app.use(errorHandler);  
  
const port = process.env.NODE_ENV === 'production' ? (process.env.PORT || 80) :  
4000;  
app.listen(port, () => console.log('Server listening on port ' + port));
```

Step 9: Running and Verification

1. **Start Dev Mode:** Run `npm run start:dev`.
2. **Access Documentation:** Navigate to `http://localhost:4000/api-docs` to interact with your API.
3. **Testing:** Follow the Postman instructions to Register, Verify via Ethereum Email, and Authenticate to receive your JWT.